2 Introduction

- What is SWIG?
- Why use SWIG?
- A SWIG example
    - SWIG interface file
    - The swig command
    - Building a Perl5 module
    - Building a Python module
    - Shortcuts
- Supported C/C++ language features
- Non-intrusive interface building
- Incorporating SWIG into a build system
- Hands off code generation
- SWIG and freedom

2.1 What is SWIG?

SWIG is a software development tool that simplifies the task of interfacing different languages to C and C++ programs. In a nutshell, SWIG is a compiler that takes C declarations and creates the wrappers needed to access those declarations from other languages including including Perl, Python, Tcl, Ruby, Guile, and Java. SWIG normally requires no modifications to existing code and can often be used to build a usable interface in only a few minutes. Possible applications of SWIG include:

- Building interpreted interfaces to existing C programs.
- Rapid prototyping and application development.
- Interactive debugging.
- Reengineering or refactoring of legacy software into a scripting language components.
- Making a graphical user interface (using Tk for example).
- Testing of C libraries and programs (using scripts).
- Building high performance C modules for scripting languages.
- Making C programming more enjoyable (or tolerable depending on your point of view).
- Impressing your friends.
- Obtaining vast sums of research funding (although obviously not applicable to the author).

SWIG was originally designed to make it extremely easy for scientists and engineers to build extensible scientific software without having to get a degree in software engineering. Because of this, the use of SWIG tends to be somewhat informal and ad-hoc (e.g., SWIG does not require users to provide formal interface specifications as you would find in a dedicated IDL compiler). Although this style of development isn't appropriate for every project, it is particularly well suited to software development in the small; especially the research and development work that is commonly found in scientific and engineering projects.

2.2 Why use SWIG?

As stated in the previous section, the primary purpose of SWIG is to simplify the task of integrating C/C++ with other programming languages. However, why would anyone want to do that? To answer that question, it is useful to list a few strengths of C/C++ programming:

- Excellent support for writing programming libraries.
- High performance (number crunching, data processing, graphics, etc.).
- Systems programming and systems integration.
- Large user community and software base.

Next, let's list a few problems with C/C++ programming

- Writing a user interface is rather painful (i.e., consider programming with MFC, X11, GTK, or any number of other libraries).
- Testing is time consuming (the compile/debug cycle).
- Not easy to reconfigure or customize without recompilation.
- Modularization can be tricky.
- Security concerns (buffer overflow for instance).

To address these limitations, many programmers have arrived at the conclusion that it is much easier to use different programming languages for different tasks. For instance, writing a graphical user interface may be significantly easier in a scripting language like Python or Tcl (consider the reasons why millions of programmers have used languages like Visual Basic if you need more proof). An interactive interpreter might also serve as a useful debugging and testing tool. Other languages like Java might greatly simplify the task of writing distributed computing software. The key point is that different programming languages offer different strengths and weaknesses. Moreover, it is extremely unlikely that any programming is ever going to be perfect. Therefore, by combining languages together, you can utilize the best features of each language and greatly simplify certain aspects of software development.

From the standpoint of C/C++, a lot of people use SWIG because they want to break out of the traditional monolithic C programming model which usually results in programs that resemble this:

- A collection of functions and variables that do something useful.
- A `main()` program that starts everything.
- A horrible collection of hacks that form some kind of user interface (but which no-one really wants to touch).

Instead of going down that route, incorporating C/C++ into a higher level language often results in a more modular design, less code, better flexibility, and increased programmer productivity.

SWIG tries to make the problem of C/C++ integration as painless as possible. This allows you to focus on the underlying C program and using the high-level language interface, but not the tedious and complex chore of making the two languages talk to each other. At the same time, SWIG recognizes that all applications are different. Therefore, it provides a wide variety of customization features that let you change almost every aspect of the language bindings. This is the main reason why SWIG has such a large user manual ;-).

2.3 A SWIG example

The best way to illustrate SWIG is with a simple example. Consider the following C code:

```
/* File : example.c */

double  My_variable  = 3.0;

/* Compute factorial of n */
int  fact(int n) {
        if (n <= 1) return 1;
```

```
        else return n*fact(n-1);
}

/* Compute n mod m */
int my_mod(int n, int m) {
        return(n % m);
}
```

Suppose that you wanted to access these functions and the global variable `My_variable` from Tcl. You start by making a SWIG interface file as shown below (by convention, these files carry a .i suffix) :

2.3.1 SWIG interface file

```
/* File : example.i */
%module example
%{
/* Put headers and other declarations here */
%}

extern double My_variable;
extern int    fact(int);
extern int    my_mod(int n, int m);
```

The interface file contains ANSI C function prototypes and variable declarations. The `%module` directive defines the name of the module that will be created by SWIG. The `%{,%}` block provides a location for inserting additional code such as C header files or additional C declarations.

2.3.2 The swig command

SWIG is invoked using the `swig` command. We can use this to build a Tcl module (under Linux) as follows :

```
unix > swig -tcl example.i
unix > gcc -c -fpic example.c example_wrap.c -I/usr/local/include
unix > gcc -shared example.o example_wrap.o -o example.so
unix > tclsh
% load ./example.so
% fact 4
24
% my_mod 23 7
2
% expr $My_variable + 4.5
7.5
%
```

The `swig` command produced a new file called `example_wrap.c` that should be compiled along with the `example.c` file. Most operating systems and scripting languages now support dynamic loading of modules. In our example, our Tcl module has been compiled into a shared library that can be loaded into Tcl. When loaded, Tcl can now access the functions and variables declared in the SWIG interface. A look at the file `example_wrap.c` reveals a hideous mess. However, you almost never need to worry about it.

2.3.3 Building a Perl5 module

Now, let's turn these functions into a Perl5 module. Without making any changes type the following (shown for Solaris):

```
unix > swig -perl5 example.i
unix > gcc -c example.c example_wrap.c \
       -I/usr/local/lib/perl5/sun4-solaris/5.003/CORE
unix > ld -G example.o example_wrap.o -o example.so            # This is for Solaris
unix > perl5.003
use example;
print example::fact(4), "\n";
print example::my_mod(23,7), "\n";
print $example::My_variable + 4.5, "\n";
<ctrl-d>
24
2
7.5
unix >
```

2.3.4 Building a Python module

Finally, let's build a module for Python (shown for Irix).

```
unix > swig -python example.i
unix > gcc -c -fpic example.c example_wrap.c -I/usr/local/include/python2.0
unix > gcc -shared example.o example_wrap.o -o _example.so
unix > python
Python 2.0 (#6, Feb 21 2001, 13:29:45)
[GCC egcs-2.91.66 19990314/Linux (egcs-1.1.2 release)] on linux2
Type "copyright", "credits" or "license" for more information.
>>> import example
>>> example.fact(4)
24
>>> example.my_mod(23,7)
2
>>> example.cvar.My_variable + 4.5
7.5
```

2.3.5 Shortcuts

To the truly lazy programmer, one may wonder why we needed the extra interface file at all. As it turns out, you can often do without it. For example, you could also build a Perl5 module by just running SWIG on the C header file and specifying a module name as follows

```
unix > swig -perl5 -module example example.h
unix > gcc -c example.c example_wrap.c \
       -I/usr/local/lib/perl5/sun4-solaris/5.003/CORE
unix > ld -G example.o example_wrap.o -o example.so
unix > perl5.003
use example;
print example::fact(4), "\n";
print example::my_mod(23,7), "\n";
print $example::My_variable + 4.5, "\n";
<ctrl-d>
24
```

```
2
7.5
```

2.4 Supported C/C++ language features

A primary goal of the SWIG project is to make the language binding process extremely easy. Although a few simple examples have been shown, SWIG is quite capable in supporting most of C++. Some of the major features include:

- Full C99 preprocessing.
- All ANSI C and C++ datatypes.
- Functions, variables, and constants.
- Classes.
- Single and multiple inheritance.
- Overloaded functions and methods.
- Overloaded operators.
- C++ templates (including member templates, specialization, and partial specialization).
- Namespaces.
- Variable length arguments.
- C++ smart pointers.

Currently, the only major C++ feature not supported is nested classes--a limitation that will be removed in a future release.

It is important to stress that SWIG is not a simplistic C++ lexing tool like several apparently similar wrapper generation tools. SWIG not only parses C++, it implements the full C++ type system and it is able to understand C++ semantics. SWIG generates its wrappers with full knowledge of this information. As a result, you will find SWIG to be just as capable of dealing with nasty corner cases as it is in wrapping simple C++ code. In fact, SWIG is able handle C++ code that stresses the very limits of many C++ compilers.

2.5 Non-intrusive interface building

When used as intended, SWIG requires minimal (if any) modification to existing C or C++ code. This makes SWIG extremely easy to use with existing packages and promotes software reuse and modularity. By making the C/C++ code independent of the high level interface, you can change the interface and reuse the code in other applications. It is also possible to support different types of interfaces depending on the application.

2.6 Incorporating SWIG into a build system

SWIG is a command line tool and as such can be incorporated into any build system that supports invoking external tools/compilers. SWIG is most commonly invoked from within a Makefile, but is also known to be invoked from from popular IDEs such as Microsoft Visual Studio.

If you are using the GNU Autotools (Autoconf/ Automake/ Libtool) to configure SWIG use in your project, the SWIG Autoconf macros can be used. The primary macro is `ac_pkg_swig`, see http://www.gnu.org/software/ac-archive/htmldoc/ac_pkg_swig.html. The `ac_python_devel` macro is also helpful for generating Python extensions. See the Autoconf Macro Archive for further information on this and other Autoconf macros.

There is growing support for SWIG in some build tools, for example CMake is a cross-platform, open-source build manager with built in support for SWIG. CMake can detect the SWIG executable and many of the target language libraries for linking against. CMake knows how to build shared libraries and loadable modules on many different operating systems. This allows easy cross platform SWIG development. It also can generate the custom commands necessary for driving SWIG from IDE's and makefiles. All of this can be done from a single cross platform input file. The following example is a CMake input file for creating a python wrapper for the SWIG interface file, example.i:

```
# This is a CMake example for Python

FIND_PACKAGE(SWIG REQUIRED)
INCLUDE(${SWIG_USE_FILE})

FIND_PACKAGE(PythonLibs)
INCLUDE_DIRECTORIES(${PYTHON_INCLUDE_PATH})

INCLUDE_DIRECTORIES(${CMAKE_CURRENT_SOURCE_DIR})

SET(CMAKE_SWIG_FLAGS "")

SET_SOURCE_FILES_PROPERTIES(example.i PROPERTIES CPLUSPLUS ON)
SET_SOURCE_FILES_PROPERTIES(example.i PROPERTIES SWIG_FLAGS "-includeall")
SWIG_ADD_MODULE(example python example.i example.cxx)
SWIG_LINK_LIBRARIES(example ${PYTHON_LIBRARIES})
```

The above example will generate native build files such as makefiles, nmake files and Visual Studio projects which will invoke SWIG and compile the generated C++ files into _example.so (UNIX) or _example.dll (Windows).

2.7 Hands off code generation

SWIG is designed to produce working code that needs no hand-modification (in fact, if you look at the output, you probably won't want to modify it). You should think of your target language interface being defined entirely by the input to SWIG, not the resulting output file. While this approach may limit flexibility for hard-core hackers, it allows others to forget about the low-level implementation details.

2.8 SWIG and freedom

No, this isn't a special section on the sorry state of world politics. However, it may be useful to know that SWIG was written with a certain "philosophy" about programming---namely that programmers are smart and that tools should just stay out of their way. Because of that, you will find that SWIG is extremely permissive in what it lets you get away with. In fact, you can use SWIG to go well beyond "shooting yourself in the foot" if dangerous programming is your goal. On the other hand, this kind of freedoom may be exactly what is needed to work with complicated and unusual C/C++ applications.

Ironically, the freedom that SWIG provides is countered by an extremely conservative approach to code generation. At it's core, SWIG tries to distill even the most advanced C++ code down to a small well-defined set of interface building techniques based on ANSI C programming. Because of this, you will find that SWIG interfaces can be easily compiled by virtually every C/C++ compiler and that they can be used on any platform. Again, this is an important part of staying out of the programmer's way---

-the last thing any developer wants to do is to spend their time debugging the output of a tool that relies on non-portable or unreliable programming features.